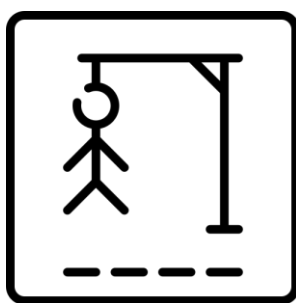




Rapport de Projet

Sujet : Réalisation du jeu du pendu en réseau

Réalisé par : Myriam MANSOURI et Ahmet KALYONCU



Sommaire

1. Introduction	3
2. Règles du jeu	3
3. Questions/Réponses.....	3
4. Extensions ajoutées	10
4.1 Chat	10
4.2 Chronomètre	11
4.3 Créer ou Rejoindre un salon	11
5. Diagramme de classe.....	11
6. Conclusion.....	11

1. Introduction

L'objectif était de réaliser en binôme le jeu du pendu en réseau avec programmation socket. Nous avons choisi Java comme langage de programmation.

Dans le premier mode de jeu, le client joue contre le serveur.

Tandis que dans le deuxième mode de jeu, les clients jouent les uns contre les autres.

2. Règles du jeu

Les règles du jeu du pendu sont simples : un mot est sélectionné aléatoirement par le serveur, et les joueurs doivent deviner les lettres qui composent ce mot. Ils ont un nombre limité de tentatives. Chaque proposition correcte révèle les lettres correspondantes dans le mot. Si toutes les lettres sont trouvées avant que les tentatives ne soient épuisées, le joueur gagne. Sinon, la partie est perdue.

3. Questions/Réponses

1. Quel service est rendu par le serveur ?

Le serveur gère le jeu, la communication entre les clients, la synchronisation des informations, et la validation des actions.

2. Que demande le client ?

Le client fait des demandes pour : se connecter au serveur et initialiser une partie, interagir avec le jeu, gérer la fin du jeu et redémarrer si nécessaire, assurer la communication avec le serveur pour recevoir et envoyer des informations en temps réel pendant la partie.

3. Que répondra le serveur à la demande du client ?

Le serveur confirme la réussite d'une action (connexion, démarrage de partie, proposition correcte). Il envoie des informations sur le mot à deviner, le nombre de tentatives restantes, et les lettres proposées. Il répond avec des messages d'erreur si des actions incorrectes sont effectuées (lettre déjà proposée, erreur de connexion, etc.). Lorsque la partie se termine, le serveur annonce la victoire ou la défaite et propose éventuellement de recommencer.

4. Qu'est-ce qui est donc à la charge du serveur ?

Gestion des connexions et des déconnexions des clients. Initialisation et suivi de l'état du jeu (mot à deviner, tentatives, etc.). Validation des actions des joueurs (propositions de lettres, etc.). Communication et synchronisation des joueurs (en

mode multijoueur). Gestion d'un système de chat permettant aux joueurs de communiquer entre eux durant la partie. Gestion de la fin de la partie et des résultats.

5. Qui du serveur ou du client fixe les règles du jeu ?

Le client définit les principales règles du jeu, notamment le nombre de tentatives et la durée de la partie.

6. Définir la structure et les champs d'un message envoyé par le client.

Avant le début de la partie :

Action	Type	Description
Choix de l'identifiant du joueur	Chaîne de caractères	Identifiant unique du joueur
Choix du mode de jeu	Entier (1 ou 2)	1 pour solo, 2 pour multijoueur
Choix du nombre de tentatives	Entier	Nombre de tentatives autorisées
Choix du temps max pour la partie	Entier	Durée maximale de la partie en secondes
Créer ou rejoindre un salon (mode multi)	Entier (1 ou 2)	1 pour créer, 2 pour rejoindre
Si créer un salon : ID du salon	Chaîne de caractères	Identifiant unique du salon à créer
Si rejoindre un salon : ID du salon	Chaîne de caractères	Identifiant du salon à rejoindre
Démarrage de la partie	Chaîne de caractères	Message "start" pour démarrer la partie

Pendant la partie :

Action	Type	Description
Proposition de lettre ou de mot	Chaîne de caractères (regex : <code>^[a-zA-Z]+\$</code>)	Le joueur envoie une lettre ou un mot sans espaces, pour deviner une lettre ou un mot.
Message de chat	String chatMessage = "CHAT:{id_joueur}:{message}"	Le joueur envoie un message dans le chat avec son identifiant et le contenu du message.

7. Définir la structure et les champs d'un message envoyé par le serveur.

Avant la partie :

Action	Format du message	Description
Choisir le mode de jeu	"Quel mode de jeu ? (1 = solo, 2 = multi)"	Le serveur demande au joueur de choisir entre le mode solo (1) ou multijoueur (2).
Nombre de tentatives	"Combien de tentatives ?"	Le serveur demande au joueur de spécifier le nombre de tentatives autorisées pour le mode solo.
Temps maximum	"Combien de temps max pour la partie ?"	Le serveur demande au joueur de définir la durée maximale de la partie en secondes pour le mode solo.
Mode multijoueur : créer ou rejoindre	"Créer ou rejoindre un salon ? (1 = créer, 2 = rejoindre)"	Le serveur demande au joueur de choisir entre créer un salon ou rejoindre un salon existant.
Créer un salon : choix de l'ID	"Choisissez un ID pour la partie"	Le serveur demande au joueur de spécifier un identifiant unique pour le salon dans le mode multijoueur.
Rejoindre un salon : demander ID	"Entrez l'ID de la partie à rejoindre"	Le serveur demande au joueur de fournir l'ID du salon à rejoindre dans le mode multijoueur.
Démarrage de la partie	"Démarrage de la partie"	Le serveur informe que la partie commence une fois que les règles ont été définies.

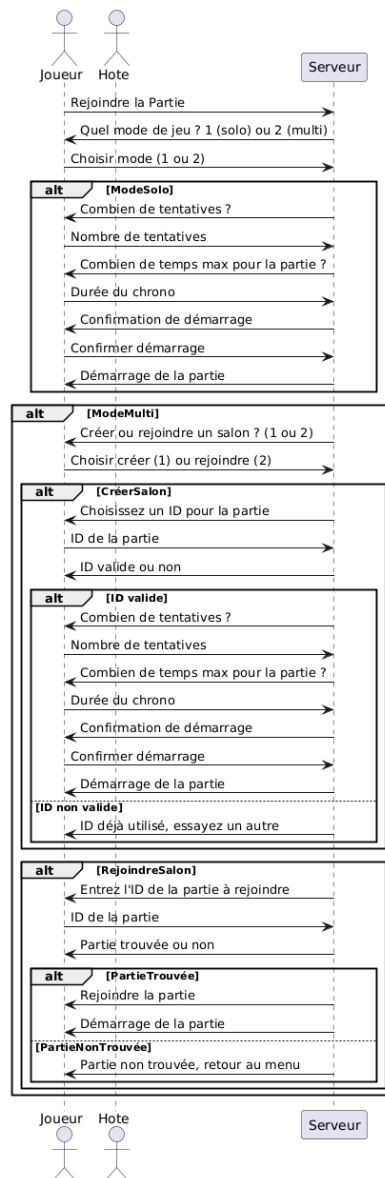
Pendant la partie :

Action	Condition	Message envoyé	Description
Gagner	Si un joueur a gagné (message égal à "gagne")	"gagne" et "Vous avez gagné ! Le mot était [mot]"	Le serveur informe le joueur gagnant qu'il a remporté la partie, en précisant le mot à deviner.
Perdre	Si la partie est perdue (message égal à "perdu")	"perdu" et "Le mot était [mot]"	Le serveur informe tous les joueurs que la partie est perdue et affiche le mot à deviner.
Perdre (partie)	Si un joueur a perdu la partie (message égal à "perduPartie")	"perduPartie" et "Le mot était [mot]"	Le serveur informe un joueur qu'il a perdu et donne le mot à deviner.
Lettre déjà utilisée	Si une lettre a déjà été utilisée (message égal à "usedLetter")	"Lettre déjà utilisé, veuillez en saisir une autre." et "Nombre vie : [nbTentatives], état actuel : [motDevine]"	Le serveur informe les joueurs que la lettre proposée a déjà été utilisée et affiche l'état du mot et le nombre de tentatives restantes.
Temps écoulé	Si le chrono arrive à zéro (message égal à "chronoA0")	"perdu"	Le serveur annonce la fin de la partie, car le temps est écoulé.
Mise à jour de l'état	Dans tous les autres cas	"État actuel : [message]" et "Nombre vie : [nbTentatives]"	Le serveur envoie une mise à jour de l'état du jeu et le nombre de tentatives restantes aux joueurs.

8. et 9.

Elaborer la séquençement des messages échangés entre le client et le serveur.
Définir précisément les diagrammes qui décrivent : une partie gagnée, perdue,
abandonnée par le client.

Diagramme de séquence : Avant le début de la partie



Connexion et choix du mode de jeu

Dans un premier temps, le joueur interagit avec le serveur pour rejoindre une partie. Le serveur demande au joueur de choisir entre deux modes de jeu : **mode solo** ou **mode multijoueur**. Le joueur transmet sa décision au serveur :

- Si le joueur choisit le mode **solo**, le serveur initie la configuration de la partie en fonction de paramètres définis par le joueur.
- Si le joueur opte pour le mode **multijoueur**, le processus diverge en fonction de s'il souhaite créer ou rejoindre un salon.

Mode Solo

Dans ce mode, le joueur configure les paramètres de la partie directement avec le serveur.

1. **Paramètres de jeu :**
 - Le serveur demande au joueur le **nombre de tentatives** pour la partie. Le joueur fournit une valeur.
 - Le serveur demande ensuite la **durée maximale** (en temps) pour la partie. Le joueur transmet également cette information.
2. **Confirmation et démarrage :**
 - Le serveur envoie un récapitulatif des paramètres choisis.
 - Une fois que le joueur confirme le démarrage, le serveur initialise la partie.

Mode Multijoueur

Dans ce mode, le joueur peut choisir de **créer un salon** ou de **rejoindre un salon existant**. Le serveur adapte ses interactions selon le choix du joueur.

Créer un salon :

1. **Création de l'ID :**
 - Le serveur demande au joueur de définir un **ID** pour identifier le salon.
 - Le joueur propose un ID, que le serveur valide. Si l'ID est déjà utilisé, le serveur demande au joueur d'en proposer un autre.
2. **Paramètres de jeu :**
 - Une fois l'ID validé, le processus de configuration est similaire au mode solo : le joueur définit le nombre de tentatives et la durée maximale de la partie.

3. **Confirmation et démarrage :**

- Le serveur envoie une confirmation des paramètres choisis.
- Le joueur confirme, et le serveur attend d'autres joueurs avant de démarrer la partie.

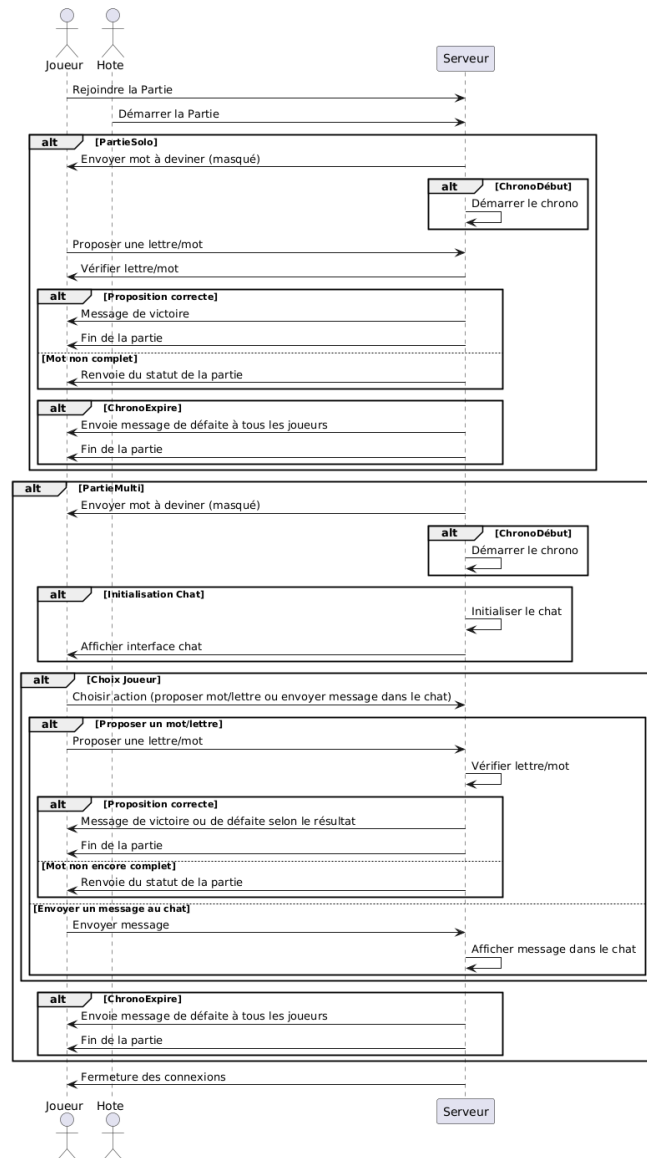
Rejoindre un salon :

1. **Recherche de partie :**

- Le serveur demande au joueur d'entrer l'ID du salon qu'il souhaite rejoindre.
- Le joueur fournit un ID. Le serveur vérifie si un salon avec cet ID existe.

2. **Résultat de la recherche :**

- Si le salon est trouvé, le serveur permet au joueur de le rejoindre. Une fois tous les joueurs connectés, le serveur lance la partie.
- Si aucun salon avec cet ID n'est trouvé, le serveur informe le joueur et le renvoie au menu principal.



4. Extensions ajoutées

4.1 Chat

Une fonctionnalité de chat a été ajoutée pour permettre aux joueurs de communiquer en temps réel pendant les parties. Cela favorise l'interactivité entre les joueurs.

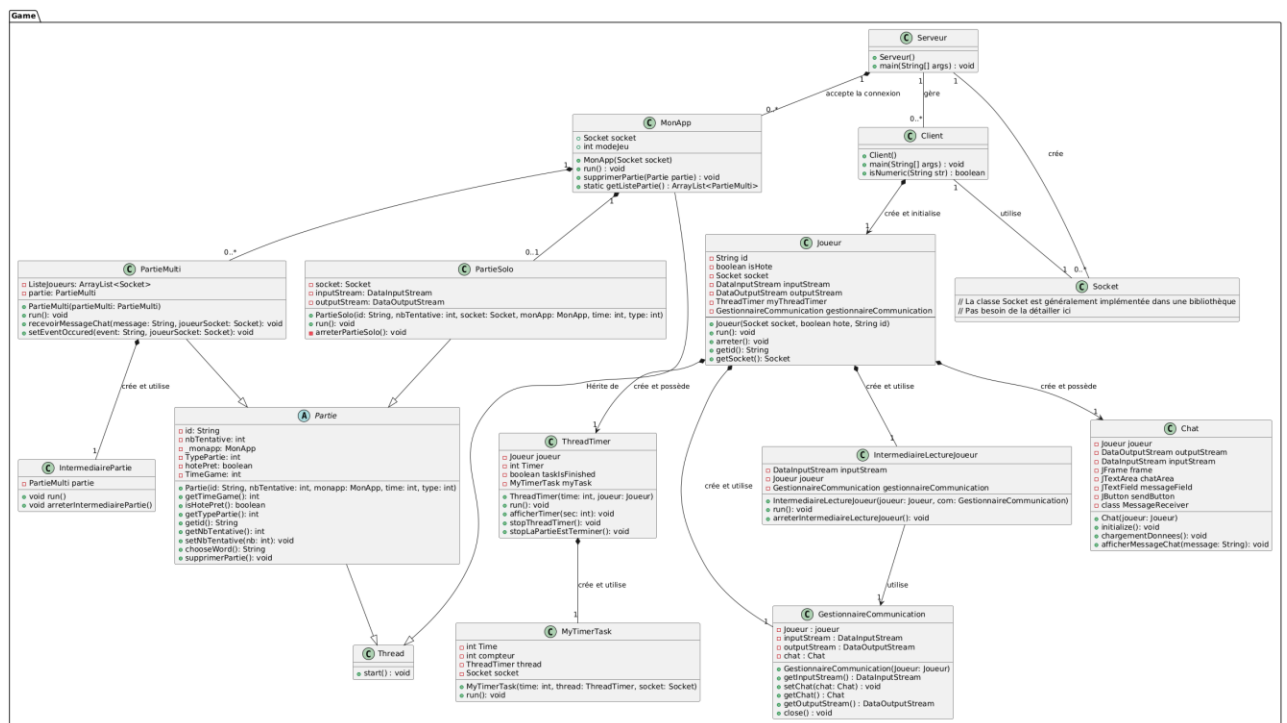
4.2 Chronomètre

Un chronomètre a été intégré afin de limiter le temps de jeu. C'est le joueur, avant de commencer à jouer, qui choisit le temps souhaité. Cette fonctionnalité ajoute un élément de pression et rend le jeu plus dynamique.

4.3 Créer ou Rejoindre un salon

Les joueurs peuvent créer ou rejoindre des salons spécifiques pour jouer avec des amis. Pour créer un salon le joueur doit lui donner un identifiant et pour rejoindre un salon le joueur doit entrer un identifiant existant. Chaque salon est géré indépendamment par le serveur.

5. Diagramme de classe



6. Conclusion

Ce projet a permis de développer une application en réseau intégrant plusieurs fonctionnalités avancées. Nous avons appris à gérer la communication entre clients et serveur, à synchroniser des données en temps réel, et à offrir une expérience interactive aux utilisateurs. Les extensions ajoutées apportent une valeur supplémentaire au jeu, et plusieurs pistes d'améliorations pourraient encore enrichir ce projet. Nous avons acquis des

compétences grâce à la réalisation de ce projet qui nous a permis de mettre en pratique ce qui a été vu en cours.